

Cryptography Evolution — From RSA to X25519

A simple guide to every algorithm change we made, why we made it, and how our security compares to WhatsApp.

1. Key Exchange: RSA → ECDH P-256 → X25519

What it does

When two users want to chat, they need a shared secret key. Key exchange lets them agree on one without ever sending it over the network.

The journey

Version	Algorithm	How it worked
v1	RSA-OAEP 2048	Sender encrypts a random chat key with recipient's RSA public key. Recipient decrypts with private key.
v2	ECDH P-256	Both sides do a Diffie-Hellman exchange on the P-256 curve. Shared secret is derived, no key is ever transmitted.
v3 (now)	X25519	Same idea as ECDH, but on Curve25519 instead of P-256.

Why each change?

RSA → ECDH P-256:

- RSA key exchange means the sender *chooses* the chat key and encrypts it. If the sender's system is compromised, the key is known.
- ECDH means *both* parties contribute randomness — neither side alone decides the key.
- ECDH keys are 32 bytes vs RSA's 256 bytes — much smaller, faster.

ECDH P-256 → X25519:

- P-256 was designed by NIST. NIST has had trust issues (they standardised a backdoored random number generator called Dual_EC_DRBG).
- X25519 was designed by Daniel Bernstein, independently reviewed, and is simpler to implement safely.
- P-256 requires careful implementation to avoid timing attacks. X25519 is constant-time by design — every operation takes the same time regardless of the key, so attackers can't learn anything by measuring how long things take.
- X25519 is ~40% faster than P-256.

2. Signatures: None → Ed25519

What it does

Signatures prove that a message was actually sent by the person who claims to have sent it. Without signatures, the server (or an attacker) could modify or forge messages.

Before

No signatures at all. The server could theoretically alter message content in transit and neither party would know.

Now (Ed25519)

- Every user gets a long-term Ed25519 signing key at signup.
- Every message is signed before encryption.
- The recipient verifies the signature after decryption.
- A green ✓ appears if verification passes, orange ! if it fails.

Why Ed25519?

- It's built on the same Curve25519 family as X25519, so one well-audited curve handles both jobs.
- Signatures are only 64 bytes (RSA signatures would be 256 bytes).
- Signing and verification are extremely fast.
- No known practical attacks exist against it.

3. Password KDF: PBKDF2 → Argon2id

What it does

Your password is used to encrypt your master key. A Key Derivation Function (KDF) makes it expensive to guess passwords by brute force.

Before (PBKDF2)

PBKDF2 runs SHA-256 many times in a loop. It's slow on a CPU, but GPUs can run thousands of SHA-256 hashes in parallel. An attacker with a \$5,000 GPU rig can try billions of passwords per day.

Now (Argon2id)

Argon2id requires **64 MB of RAM per guess**. GPUs have fast cores but limited per-core memory. This makes GPU/ASIC attacks roughly 1,000x more expensive.

Property	PBKDF2	Argon2id
Memory needed per guess	~0	64 MB
GPU-friendly?	Yes (bad for us)	No (good for us)
Standardised?	NIST SP 800-132	RFC 9106, won Password Hashing Competition (2015)

Property	PBKDF2	Argon2id
Side-channel resistance	Basic	Argon2id combines Argon2i (side-channel safe) + Argon2d (GPU-hard)

Our parameters: 64 MiB memory, 3 iterations, parallelism 1, 32-byte output.

4. Forward Secrecy: None → X3DH-style Prekeys

What it does

Forward secrecy means: even if your long-term private key is stolen *tomorrow*, messages sent *today* cannot be decrypted.

Before

Every chat used a key derived from both users' long-term static keys. If either long-term key was ever compromised, **all past and future messages** in that chat could be decrypted.

Now (Ephemeral Prekeys)

- At signup, each user generates 10 one-time X25519 prekeys.
- When Alice starts a chat with Bob, she fetches one of Bob's prekeys from the server.
- Alice generates a fresh ephemeral key and combines it with Bob's prekey to derive a unique session key.
- Bob's prekey is deleted from the server after use — it can never be reused.
- Even if Alice's or Bob's long-term key leaks later, this session key cannot be reconstructed.

This is modelled on Signal's X3DH (Extended Triple Diffie-Hellman) protocol.

5. Device Trust: None → Device Identity Keys

What it does

Prevents a man-in-the-middle (MITM) attack where the server substitutes a fake public key when you upload yours.

Before

When you uploaded your public key, there was no proof it really came from your device. A compromised server could swap in its own key and read all your messages.

Now

- Each device generates its own Ed25519 identity key (stored locally).
- When uploading your X25519 public key, the device signs it with its device key.
- Other devices (or future audit tools) can verify the signature to confirm the key wasn't tampered with.

Important: This is NOT multi-device support

Currently we store only **one** device key per user. If you log in from a second device, it overwrites the first. True multi-device would need a per-device table, per-device encryption, and a device linking protocol. The device key today exists purely to **prove key authenticity** — not to manage multiple devices.

6. What Stayed the Same

Component	Algorithm	Why kept
Symmetric encryption	AES-256-GCM	Gold standard. 256-bit key, authenticated encryption (detects tampering). No reason to change.
Key derivation (shared secret → chat key)	HKDF-SHA-256	Industry standard (RFC 5869). Used by Signal, TLS 1.3, and WhatsApp.
IV generation	96-bit random per message	Standard for AES-GCM. Random IV ensures no two messages produce the same ciphertext.

Security Comparison: Us vs WhatsApp

WhatsApp uses the Signal Protocol (designed by Open Whisper Systems). Here's how we compare:

Where we match WhatsApp

Feature	WhatsApp	Us
End-to-end encryption	✔ AES-256-GCM	✔ AES-256-GCM
Key agreement	✔ X25519	✔ X25519
Key derivation	✔ HKDF-SHA-256	✔ HKDF-SHA-256
Message signatures	✔ Ed25519	✔ Ed25519
Forward secrecy (prekeys)	✔ X3DH	✔ X3DH-style
Zero-knowledge server	✔ Server can't read messages	✔ Server can't read messages
Password hashing	✔ (handled by device OS)	✔ Argon2id

Where we are BETTER than WhatsApp

Feature	WhatsApp	Us
Open source	✗ Closed source (server + client)	✔ Fully open source — anyone can audit
Device identity binding	✗ Key uploads are not device-signed	✔ Each device signs key uploads with its own Ed25519 key
Password protection	Relies on phone OS keychain	Argon2id (memory-hard, GPU-resistant) protects master key

Feature	WhatsApp	Us
Metadata visibility	✗ WhatsApp sees who talks to whom, when, and how often	Our server sees less metadata (no phone numbers, no contact graph sync)
Trust model	Trust Meta (Facebook) won't peek	Trust is unnecessary — code is auditable, server is zero-knowledge

Where WhatsApp is BETTER than us

Feature	WhatsApp	Us
Double Ratchet	✓ Every single message uses a new key (break-in recovery + per-message forward secrecy)	✗ We use one session key per chat session, not per message
Key verification UX	✓ QR code / safety number comparison between devices	✗ No user-facing verification mechanism yet
Sealed sender	✓ Server doesn't know who sent a message (metadata hiding)	✗ Server sees sender ID on each message
Multi-device sync	✓ Seamless multi-device with per-device encryption	⚠ Basic device identity keys exist, but no multi-device key sync yet
Backup encryption	✓ Encrypted cloud backups with user password	⚠ Recovery key exists but no encrypted cloud backup
Scale & audit	Battle-tested by 2+ billion users, audited by multiple firms	New codebase, not yet independently audited
Disappearing messages	✓ Built-in message expiry	✗ Not implemented

Summary

Our app uses the **same core cryptographic primitives** as WhatsApp (X25519, Ed25519, AES-256-GCM, HKDF). The biggest gap is the **Double Ratchet** — WhatsApp generates a new encryption key for every single message, so compromising one message key reveals nothing about any other message. We derive one key per chat session, which is good but not as strong.

The biggest advantage we have is **full transparency** — the code is open source, the server is zero-knowledge, and users don't have to trust a corporation to not peek at their data.

Last updated: March 2026 — feature/crypto-upgrade-v2 branch